

Tema 3 — Funciones

Laboratorio Guiado

Objetivos	Practicar la creación de funciones en Python: definición con def, llamada a funciones, parámetros, argumentos, return, valores por defecto, alcance de variables, argumentos variables, anotaciones de tipo y docstrings.
Material necesario	Terminal, Python 3.13, venv del Tema03, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en ~/Curso_Python/Tema03.

Laboratorio 1. Definición básica con def y llamada a funciones

1. Actualizar el repositorio y abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema03
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero 01_def_y_llamadas.py. Revisar el docstring inicial del script y observar que explica el objetivo general del laboratorio.

```
"""
Tema 3 - Funciones
Laboratorio 1: definición básica con def y llamada a funciones.

Objetivo:
    Entender que definir una función no la ejecuta.
    La función solo se ejecuta cuando se invoca usando su nombre y paréntesis.
"""
```

3. Revisar una primera función sin parámetros. La función contiene un mensaje interno y usa print() para mostrarlo por pantalla.

```
def saludar_operador():
    """Muestra un mensaje básico de inicio de sesión de soporte."""
    mensaje = "Sesión de soporte iniciada"
    print(mensaje)
```

4. Revisar una función con un parámetro. El parámetro servicio recibe un valor concreto en cada llamada.

```
def mostrar_estado(servicio):
    """Muestra por pantalla el nombre de un servicio que se está comprobando."""
    print(f"Comprobando servicio: {servicio}")
```

5. Analizar una función que devuelve un valor con return. En este caso, la función clasifica el uso de CPU y devuelve una cadena reutilizable.

```
def estado_cpu(uso):  
    """  
    Clasifica el uso de CPU en un estado operativo.  
  
    Args:  
        uso: Porcentaje de uso de CPU.  
  
    Returns:  
        Una cadena con el estado NORMAL, ALTO o CRÍTICO.  
    """  
    if uso >= 90:  
        return "CRÍTICO"  
    if uso >= 70:  
        return "ALTO"  
    return "NORMAL"
```

6. Ejecutar las llamadas finales del script y comprobar que las funciones solo se ejecutan cuando se invocan.

```
saludar_operador()  
  
mostrar_estado("ssh")  
mostrar_estado("httpd")  
mostrar_estado("backup")  
  
uso_actual = 82  
print(f"Uso de CPU {uso_actual}% -> {estado_cpu(uso_actual)}")  
  
python 01_def_y_llamadas.py
```

Resultado esperado: se muestran los mensajes de inicio, la comprobación de servicios y la clasificación del uso de CPU.

Laboratorio 2. Parámetros, argumentos y return

1. Abrir el fichero 02_parametros_return.py. El objetivo es diferenciar los datos que recibe una función y el valor que devuelve.
2. Revisar la función calcular_iva(). Los nombres base y porcentaje son parámetros; los valores enviados en cada llamada son argumentos.

```
def calcular_iva(base, porcentaje):  
    """  
    Calcula el importe del IVA aplicado sobre una base.  
  
    Args:  
        base: Importe base.  
        porcentaje: Porcentaje de IVA.  
  
    Returns:  
        Importe del impuesto calculado.  
    """  
    impuesto = base * porcentaje / 100  
    return impuesto
```

3. Revisar una segunda función que calcula y devuelve un precio final. La función no imprime: devuelve un dato que el programa puede seguir usando.

```
def calcular_descuento(precio, descuento):  
    """Calcula el precio final aplicando un porcentaje de descuento."""  
    return precio - (precio * descuento / 100)
```

4. Comparar con una función que solo muestra información con print(). Al no tener return explícito, Python devolverá None.

```
def registrar_evento(texto):  
    """  
    Muestra un evento por pantalla.  
  
    Esta función no devuelve ningún resultado explícito.  
    Python devolverá None automáticamente.  
    """  
    print(f"EVENTO: {texto}")
```

5. Ejecutar el bloque de llamadas y observar cómo se reutilizan los valores devueltos.

```
iva = calcular_iva(100, 21)  
print(f"IVA calculado: {iva:.2f} €")  
  
precio_final = calcular_descuento(80, 15)  
print(f"Precio final: {precio_final:.2f} €")  
  
resultado = registrar_evento("reinicio programado")  
print("Valor devuelto por registrar_evento:", resultado)  
  
python 02_parametros_return.py
```

Resultado esperado: las funciones con return devuelven valores utilizables; registrar_evento() imprime un mensaje, pero su resultado es None.

Laboratorio 3. Devolución múltiple y alcance de variables

1. Abrir el fichero 03_devolucion_multiple_y_alcance.py. Este laboratorio muestra cómo devolver varios valores y por qué las variables internas no deben usarse desde fuera.
2. Revisar la función resumen_tiempos(). Devuelve mínimo, máximo y media como valores relacionados.

```
def resumen_tiempos(valores):  
    """Calcula el mínimo, máximo y promedio de una lista de tiempos."""  
    minimo = min(valores)  
    maximo = max(valores)  
    media = sum(valores) / len(valores)  
    return minimo, maximo, media
```

3. Recuperar la devolución múltiple en varias variables. Python agrupa internamente los valores en una tupla.

```
tiempos_respuesta = [12, 9, 15, 11]
mn, mx, avg = resumen_tiempos(tiempos_respuesta)

print("Tiempo mínimo:", mn)
print("Tiempo máximo:", mx)
print("Tiempo medio:", round(avg, 2))
```

4. Comprobar que también se pueden guardar todos los valores devueltos en una sola variable.

```
resumen = resumen_tiempos([20, 18, 25, 19])
print("Resumen como tupla:", resumen)
```

5. Revisar el alcance local. La variable ruta se crea dentro de construir_ruta() y no existe directamente fuera de la función.

```
def construir_ruta(base, fichero):
    """Construye una ruta uniendo directorio base y nombre de fichero."""
    ruta = f"{base}/{fichero}"
    return ruta

ruta_log = construir_ruta("/var/log", "app.log")
print("Ruta construida:", ruta_log)

# Descomenta la línea siguiente para comprobar el NameError.
# print(ruta)
```

6. Ejecutar el script completo.

```
python 03_devolucion_multiple_y_alcance.py
```

Resultado esperado: el alumno debe entender que los resultados salen mediante return y que las variables locales no están disponibles fuera de la función.

Laboratorio 4. Argumentos posicionales y argumentos con nombre

1. Abrir el fichero 04_argumentos_posicionales_nombre.py. El objetivo es comprobar cuándo importa el orden de los argumentos.
2. Revisar una función con tres parámetros obligatorios.

```
def crear_usuario(nombre, grupo, activo):
    """Construye una descripción simple de usuario."""
    return f"{nombre} -> {grupo} -> activo={activo}"
```

3. Ejecutar dos llamadas posicionales. La segunda no falla, pero su resultado es incorrecto desde el punto de vista lógico porque los argumentos están en mal orden.

```
print(crear_usuario("ana", "operadores", True))

# Este ejemplo no falla, pero el resultado no tiene sentido.
print(crear_usuario("operadores", "ana", True))
```

4. Revisar una función donde los argumentos con nombre mejoran la lectura.

```
def programar_tarea(nombre, hora, habilitada):  
    """Construye una descripción de una tarea planificada."""  
    return f"{nombre} a las {hora}, habilitada={habilitada}"
```

5. Ejecutar llamadas con nombre. El orden puede cambiar porque cada argumento indica explícitamente el parámetro al que corresponde.

```
print(programar_tarea(nombre="backup", hora="02:00", habilitada=True))  
print(programar_tarea(habilitada=False, hora="03:00", nombre="limpieza"))  
  
python 04_argumentos_posicionales_nombre.py
```

Resultado esperado: las llamadas con nombre son más claras cuando hay varios parámetros del mismo tipo o cuando el orden puede resultar ambiguo.

Laboratorio 5. Valores por defecto y objetos mutables

1. Abrir el fichero 05_valores_por_defecto.py. Primero se revisan parámetros con valores por defecto.

```
def crear_alerta(mensaje, nivel="INFO", notificar=False):  
    """Crea un texto de alerta con nivel y opción de notificación."""  
    texto = f"[{nivel}] {mensaje}"  
    if notificar:  
        texto += " -> enviar aviso"  
    return texto  
  
print(crear_alerta("Servicio iniciado"))  
print(crear_alerta("Disco lleno", nivel="CRITICO", notificar=True))
```

2. Revisar el caso no recomendado: una lista como valor por defecto. La lista se crea una sola vez al definir la función y puede conservar datos entre llamadas.

```
def agregar_evento_mal(evento, historial=[]):  
    """Ejemplo no recomendado: usa una lista mutable como valor por defecto."""  
    historial.append(evento)  
    return historial  
  
print("Mal construido:")  
print(agregar_evento_mal("arranque"))  
print(agregar_evento_mal("parada"))  
print(agregar_evento_mal("reinicio"))
```

3. Revisar la forma recomendada: usar None como valor por defecto y crear la lista dentro de la función cuando no se recibe una lista externa.

```
def agregar_evento(evento, historial=None):  
    """Añade un evento a una lista recibida o crea una lista temporal."""  
    if historial is None:  
        historial = []  
    historial.append(evento)  
    return historial  
  
print("Bien construido, llamadas independientes:")  
print(agregar_evento("arranque"))  
print(agregar_evento("parada"))
```

4. Comprobar que también se puede acumular en un historial externo de forma explícita y controlada.

```
print("Bien construido, historial externo controlado:")
historico_eventos = []
print(agregar_evento("arranque", historico_eventos))
print(agregar_evento("parada", historico_eventos))

python 05_valores_por_defecto.py
```

Resultado esperado: el alumno debe distinguir entre una lista temporal por llamada y una lista externa usada conscientemente como acumulador.

Laboratorio 6. *args, **kwargs y orden recomendado de parámetros

1. Abrir el fichero 06_args_kwargs.py. Este laboratorio introduce argumentos variables sin profundizar todavía en colecciones.
2. Revisar *medidas. Permite enviar cero, uno o varios valores posicionales a la función.

```
def totalizar_consumo(*medidas):
    """Suma un número variable de medidas."""
    total = 0
    for valor in medidas:
        total += valor
    return total

print("Total de consumo:", totalizar_consumo(12, 15, 9))
print("Total sin medidas:", totalizar_consumo())
```

3. Revisar **atributos. Permite recibir pares clave=valor con información variable sobre un recurso.

```
def describir_recurso(nombre, **atributos):
    """Muestra información variable de un recurso."""
    print(f"Recurso: {nombre}")
    for clave, valor in atributos.items():
        print(f"- {clave}: {valor}")

describir_recurso("srv01", cpu=4, memoria="16GB", entorno="prod")
```

- Analizar una firma combinada. Primero aparece el parámetro obligatorio, después *valores, luego un parámetro con nombre opcional y finalmente **etiquetas.

```
def registrar_metricas(servicio, *valores, unidad="ms", **etiquetas):
    """Registra métricas de un servicio usando parámetros combinados."""
    return {
        "servicio": servicio,
        "valores": valores,
        "unidad": unidad,
        "etiquetas": etiquetas,
    }

metricas = registrar_metricas("api", 12, 15, 9, unidad="ms", zona="dmz",
entorno="prod")
print(metricas)

python 06_args_kwargs.py
```

Resultado esperado: *args agrupa argumentos posicionales variables y **kwargs agrupa argumentos con nombre variables. Ambos deben usarse cuando aporten claridad.

Laboratorio 7. Anotaciones de tipo, docstrings y buenas prácticas

- Abrir el fichero 07_anotaciones_docstrings_buenas_practicas.py. El objetivo es documentar funciones de forma clara.
- Revisar una función con anotaciones de tipo y docstring completo. Las anotaciones ayudan al editor y a otros programadores, aunque Python no las fuerza automáticamente.

```
def calcular_porcentaje(valor: float, total: float) -> float:
    """
    Devuelve el porcentaje que representa valor sobre total.

    Args:
        valor: Parte que se quiere comparar.
        total: Valor total de referencia.

    Returns:
        Porcentaje calculado.
    """
    return valor * 100 / total
```

- Revisar una función pequeña que devuelve un booleano. Este tipo de función suele usarse dentro de condicionales.

```
def es_puerto_valido(puerto: int) -> bool:
    """Comprueba si un puerto TCP/UDP está en el rango válido."""
    return 1 <= puerto <= 65535
```

- Usar la función anterior dentro de otra función que genera un mensaje. Esta separación permite reutilizar la validación.

```
def generar_mensaje_puerto(puerto: int) -> str:
    """Genera un mensaje de validación para un puerto."""
    if es_puerto_valido(puerto):
        return f"Puerto {puerto} válido"
    return f"Puerto {puerto} fuera de rango"
```

5. Ejecutar el bloque final y observar también cómo se puede consultar el docstring de una función.

```
print(f"Porcentaje: {calcular_porcentaje(45, 200):.2f}%")
print(calcular_porcentaje.__doc__)

for puerto in [22, 443, 70000, 0]:
    print(generar_mensaje_puerto(puerto))

python 07_ anotaciones_docstrings_buenas_practicas.py
```

Resultado esperado: las funciones quedan más legibles cuando tienen nombre claro, una responsabilidad concreta, anotaciones y un docstring útil.

Laboratorio 8. Ejercicios adicionales integrados

1. Abrir el fichero 08_ejercicios_adicionales.py. Este fichero agrupa funciones pequeñas para practicar reutilización de código.
2. Revisar una función que normaliza texto. La entrada se limpia y se convierte a minúsculas.

```
def normalizar_usuario(usuario: str) -> str:
    """Normaliza un nombre de usuario."""
    return usuario.strip().lower()
```

3. Revisar una función con un parámetro por defecto para construir nombres de servicio.

```
def construir_nombre_servicio(nombre: str, entorno: str = "prod") -> str:
    """Construye un nombre estándar de servicio."""
    return f"{entorno}-{nombre}".lower()
```

4. Revisar una función con *valores que puede devolver None cuando no recibe datos. Esto evita calcular una media sin valores.

```
def media_valores(*valores: float) -> float | None:
    """Calcula la media de un número variable de valores."""
    if not valores:
        return None
    return sum(valores) / len(valores)
```

5. Revisar una función que combina parámetros normales, *args y un valor por defecto. Para facilitar la lectura, se muestra la conversión de puertos con un bucle explícito.

```
def resumen_servicio(nombre: str, activo: bool, *puertos: int, entorno: str = "prod") -> str:
    """Construye un resumen de servicio con puertos asociados."""
    estado = "activo" if activo else "inactivo"

    if puertos:
        puertos_convertidos = []
        for puerto in puertos:
            puertos_convertidos.append(str(puerto))
        puertos_txt = ", ".join(puertos_convertidos)
    else:
        puertos_txt = "sin puertos"

    return f"{entorno}:{nombre} -> {estado} -> {puertos_txt}"
```


6. Ejecutar las llamadas finales y modificar algunos valores para comprobar los resultados.

```
print(normalizar_usuario("  Admin.Sistemas  "))
print(construir_nombre_servicio("api"))
print(construir_nombre_servicio("api", entorno="dev"))
print(media_valores(10, 8, 9))
print(media_valores())
print(resumen_servicio("nginx", True, 80, 443, entorno="prod"))

python 08_ejercicios_adicionales.py
```

Resultado esperado: el alumno debe identificar funciones pequeñas, valores por defecto, *args, None y reutilización de funciones en ejemplos sencillos.

Ejercicios propuestos

1. Crear un script src/calcula_area_rectangulo.py con una función area_rectangulo(base, altura) que devuelva el área y otra función mostrar_resultado() que imprima el mensaje final.
2. Crear un script src/estado_servicio.py con una función estado_servicio(nombre, activo=True) que devuelva un texto con el nombre del servicio y su estado.
3. Crear un script src/promedio_metricas.py con una función promedio(*valores) que devuelva None si no recibe valores y la media si recibe uno o más números.
4. Crear un script src/formatea_usuario.py con una función normalizar_usuario(usuario: str) -> str que elimine espacios, convierta a minúsculas y sustituya espacios internos por guiones bajos.
5. Crear un script src/resumen_backup.py con una función resumen_backup(nombre, *rutas, comprimido=False) que devuelva un texto indicando el nombre del backup, las rutas incluidas y si se comprimirá.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python/Tema03</code> .
El script no se encuentra	La terminal no está situada en el directorio del tema.	Ejecutar <code>cd ~/Curso_Python/Tema03</code> y comprobar <code>ls -la</code> .
NameError al llamar a una función	La función no se ha definido antes de llamarla o el nombre no coincide.	Revisar el orden del código y comprobar mayúsculas, minúsculas y guiones bajos.
TypeError por argumentos	La llamada no coincide con la firma de la función.	Comprobar número de argumentos, argumentos por nombre y valores por defecto.
La función imprime pero no devuelve nada	La función usa <code>print()</code> , pero no <code>return</code> .	Usar <code>return</code> cuando el resultado deba reutilizarse después.
Un valor local no existe fuera de la función	La variable tiene alcance local.	Devolver el valor con <code>return</code> y asignarlo a una variable externa.
Resultados acumulados inesperados en listas	Se ha usado un objeto mutable como valor por defecto.	Usar <code>None</code> como valor por defecto y crear la lista dentro de la función.